深度学习 CHAP6_RNN 实验报告

2352473 蒋东珣

一、实验背景

本实验的背景目的在于构建一个能够学习汉字序列统计规律的生成模型，使其在给定少量前缀信息（例如一个起始字或起始标记）的条件下，自动生成后续文本，从而形成结构连贯的诗句。它关注的是如何用概率建模的方式刻画“前文决定后文”的语言现象，把自然语言生成问题转化为“在上下文条件下预测下一字符分布”的可优化目标，并通过迭代预测实现整段文本的自动续写。

二、RNN，LSTM，GRU 模型

RNN（Recurrent Neural Network，循环神经网络）是一类专门处理序列数据的神经网络结构，核心思想是同一套参数在时间维度上反复使用，让模型在读入第 t 个输入 x_t 时，不仅依赖当前输入，还依赖之前的隐藏状态 h_{t-1}，再由 h_t 产生输出 y_t 或最终输出。因为隐藏状态会把过去信息压缩到一个固定维度的向量里，所以 RNN 天然适合语言建模、时间序列预测、序列标注等任务。在训练上通常使用 BPTT（Backpropagation Through Time，时间反向传播），把网络在时间上展开成一条很深的网络来求梯度；这也带来经典问题：序列一长，梯度在多次链式相乘后要么迅速衰减（梯度消失），导致模型难以学习长期依赖；要么迅速变大（梯度爆炸），导致训练不稳定。实际工程里常见的做法包括梯度裁剪、合理初始化、改用门控结构（LSTM/GRU）等。torch.nn.utils.clip_grad_norm(...) 就是在做梯度裁剪，缓解梯度爆炸；而生成诗句这类任务，本质上是根据前面字符预测下一个字符的序列建模，普通 RNN 能做，但长程依赖和稳定训练方面通常不如门控 RNN。

LSTM（Long Short-Term Memory）是为了解决普通 RNN 难以学习长期依赖而提出的门控循环结构，它在隐藏状态之外引入了细胞状态 c_t，并用门控机制控制信息的写入、遗忘和输出，从而让梯度可以沿着较为“线性”的路径在时间上传播，显著缓解梯度消失。LSTM 通常包含三个门：遗忘门 f_t 决定从旧记忆 c_{t-1} 中保留多少；输入门 i_t 决定把当前候选记忆 写入多少；输出门 o_t 决定从当前记忆c_t输出多少到隐藏状态 h_t。这里的关键是 c_t 的更新有一条加法通路，相比普通 RNN 全是乘法复合，更利于梯度传播；门控又能让网络选择性记忆或忘记，从而在长句子、长时间序列里保留有效上下文。在 rnn.py 里实际使用的是 nn.LSTM(..., num_layers=2, batch_first=True)，也就是两层堆叠的 LSTM，并且输入输出张量按 (batch, seq, feature) 组织；堆叠层数会增强表达能力，但也更依赖正则化、学习率和梯度裁剪等手段来稳定训练。整体流程是：字符索引先经过 Embedding 变成向量序列，再喂给 LSTM 得到每个时间步的输出表示，最后通过线性层映射到词表大小并做 LogSoftmax，用 NLLLoss 训练成下一个字符的条件概率模型。

GRU（Gated Recurrent Unit）同样是门控循环网络，但结构比 LSTM 更简洁：它把“细胞状态”和“隐藏状态”合并成一个状态向量，通常只使用两个门——更新门 z_t 和重置门 (r_t)。更新门控制“保留旧状态 vs 接受新信息”的比例，重置门控制在生成候选状态时要忘掉多少过去。可以看到 GRU 也有一条类似加法插值的更新路径（旧状态与候选状态的加权平均），因此同样能缓解梯度消失；但因为少了一个门和独立的 c_t，参数更少、计算更快、实现更简单，很多任务上性能与 LSTM 接近甚至更好，尤其在数据量中等、需要更高训练/推理效率时常被优先考虑。对比上，LSTM 更可控、容量更强，长程依赖很强或需要更精细记忆管理的任务有时更占优；GRU 更轻量，在资源受限或希望更快迭代时更常用；

普通 RNN 则通常作为教学或简单基线，在长序列任务上往往不如前两者稳定和有效。

三、诗歌生成过程

诗歌生成可以理解为字符级序列建模 + 自回归生成。一首诗被当成一个按时间顺序展开的字符序列，模型在读到第t个字符时，输出一个下一字符的概率分布（更准确说是对数概率分布）；训练阶段用真实的下一字符作为监督信号，让模型把整首诗的每一个位置都学成一个分类问题。生成阶段则把模型自己的预测当作下一步输入不断续写，直到输出结束标记或达到长度上限为止。

数据进入模型前会先完成离散化：先准备一个字符表，把训练集中出现过的每个字符映射为一个整数编号。这样，一首诗就能表示为整数序列。为了让模型学会何时开始、何时结束，通常会人为加上一个开始标记和一个结束标记，训练时把开始标记 + 正文 + 结束标记作为完整序列使用；生成时就能从开始（或指定起始字）出发，遇到结束标记就停止。这里的监督信号构造方式是经典的序列错位：输入是原序列，标签是把序列整体左移一位得到的序列，也就是让模型在每个时间步都去预测紧随其后的那个字符。

模型内部第一步是嵌入（embedding）：整数编号本身没有语义距离，直接喂给循环网络效果差，因此会先通过一个可训练的查表层把每个字符编号映射成一个连续向量。向量矩阵在初始化时使用了均匀随机值拷贝进去，之后在训练过程中会被梯度更新，使得常一起出现或语义/结构相近的字符在向量空间里逐渐形成更合适的表示。嵌入之后，序列从长度为 T 的整数变成长度为 T 的向量序列，每个时间步是一个固定维度的实数向量。

接着是两层 LSTM。LSTM 之所以适合这类任务，是因为它通过门控与记忆单元在长序列上更稳定，能在一定程度上保留更久的上下文信息。实现上，输入张量会被组织为三维：批大小、时间步、特征维（也就是嵌入维度）。虽然训练脚本里往往是“每次只送一首诗进去”，但仍然会把它包装成批大小为 1 的形式，时间步就是这首诗的长度。进入 LSTM 时会显式提供初始隐藏状态和初始记忆单元，通常全置零；两层 LSTM 对应这两个状态的第一维长度为 2（层数），第二维是批大小，第三维是隐藏维度。LSTM 的输出是“每个时间步一个隐藏向量”的序列，因此输出张量仍然保留时间步维度，表示模型在每个位置读完当前字符后形成的上下文表示。

为了把“隐藏向量”变成“对字符表的预测”，会把所有时间步的隐藏向量展平成二维矩阵，把每个时间步都当成一次独立的分类样本，再通过一个线性层把隐藏维度投影到“字符表大小”的维度上得到打分（logits）。这里额外加了一个 ReLU，会把负值截断为 0，使得线性层输出变成非线性变换后的特征；严格来说，语言模型更常见的是线性层直接接 softmax，ReLU 可能会损失一部分表达（因为它会丢掉符号信息），但在小作业里仍然能训练出可用结果。最后用对数形式的 softmax 得到每个字符作为“下一字符”的对数概率，这样就能与负对数似然损失函数直接配合，计算每个时间步预测真实下一字符的损失并累加。

训练时的损失计算可以这样理解：一首诗长度为 T，就会产生 T 次预测下一字符的分类任务，标签是长度为 T 的整数序列（每个位置是正确的下一字符编号）。损失函数会把每个时间步的交叉熵加起来（或求平均，取决于具体设置），得到这首诗的误差。训练脚本再把一个批次中多首诗的损失求平均，做一次反向传播更新参数。为了防止循环网络常见的梯度爆炸问题，反向传播后会对梯度范数进行裁剪，把过大的梯度缩到阈值以内，这能显著提升训练稳定性。优化器使用自适应或近似自适应的更新规则（例如 RMSprop），在这种序列任务上通常比朴素 SGD 更容易收敛。

生成阶段则把训练好的模型当作条件概率分布生成器。给定一个起始字（或起始标记），先把当前已生成的前缀整体送入模型，模型会在每个时间步给出下一字符分布；但真正需要的是前缀最后一个位置对应的下一步分布，因为那代表在当前整段上下文条件下，下一字符

最可能是什么。实现里会取最后一个时间步的输出分布，然后用最大概率选择（argmax）得到一个确定的下一个字符，再把它拼接到前缀末尾，形成更长的前缀，继续重复这个过程。这种做法等价于贪心解码，优点是简单稳定、结果可复现，缺点是多样性不足、容易陷入局部最优；如果改成按概率采样或加入温度系数，生成会更发散但也更像“创作”。循环终止条件通常是生成到结束标记，或者超过最大长度强制停止，避免模型迟迟不输出结束标记导致无限生成。

四、生成诗歌

（1）Tensorflow 版本

训练过程：

```
epoch 0 step 0 : loss 8.821163177490234
epoch 0 step 50 : loss 6.770924091339111
epoch 0 step 100 : loss 6.545600414276123
epoch 0 step 150 : loss 6.499835014343262
epoch 0 step 200 : loss 6.6115546226501465
epoch 0 step 250 : loss 6.569016933441162
epoch 0 step 300 : loss 6.524219989776611
epoch 0 step 350 : loss 6.58845329284668
epoch 0 step 400 : loss 6.5706987380981445
epoch 1 step 0 : loss 6.5540452003479
epoch 1 step 50 : loss 6.464052677154541
epoch 1 step 100 : loss 6.341538906097412
epoch 1 step 150 : loss 6.21603536605835
epoch 1 step 200 : loss 6.199273586273193
epoch 1 step 250 : loss 6.151977062225342
epoch 1 step 300 : loss 6.121182441711426
epoch 1 step 350 : loss 5.917440891265869
epoch 1 step 400 : loss 5.909659385681152
epoch 2 step 0 : loss 6.030253887176514
epoch 2 step 50 : loss 5.943791389465332
epoch 2 step 100 : loss 5.892157077789307
epoch 2 step 150 : loss 5.9681620597839355
epoch 2 step 200 : loss 5.8543400764465533
epoch 2 step 250 : loss 5.807015419006348
epoch 2 step 300 : loss 5.828242301940918
...
epoch 9 step 250 : loss 5.129788398742676
epoch 9 step 300 : loss 5.059021472930908
epoch 9 step 350 : loss 5.058689117431641
epoch 9 step 400 : loss 5.106410503387451
```

生成诗歌：一年年日日，春日不堪归。

解读（deepseek）：这句诗“一年年日日，春日不堪归”以极为简练的语言，道出了时间无尽与春光易逝之间深刻的矛盾与无奈。开篇“一年年日日”通过年与日的双重叠加，既勾勒出岁月循环往复的漫长感，又暗含着每日每时都在悄然流逝的细微刻度，仿佛时间在宏观与微观两个维度上都压得人喘不过气来。而“春日不堪归”则将笔触转向本该充满生机与希望的春天，却用“不堪归”三字将一切美好都染上了愁绪——或许是春光太过短暂令人不忍面对它的离去，又或许是春光虽至而人心已倦，连归来的春天都成为一种难以承受的负担。整句诗由此营造出一种惆怅而苍凉的意境，时间看似漫长却无情流逝，春天看似美好却徒增伤感，在这种矛盾之中，诗人对光阴易逝、人生易老的深切感慨跃然纸上。语言上采用重字回环的手法，“一年年”与“日日”相叠，形成缓慢而沉重的节奏，仿佛每一次重复都在敲击着读者的心扉，虽不事雕琢，却以最朴素的方式传达出最普遍的人生体验。

（2）Pytorch 版本

```
******************************
epoch   29 batch number 340 loss is:  6.104559421539307
finish  save model
prediction [10, 16, 16, 15, 35, 0, 5, 7, 179, 179, 32, 1, 7, 6, 4, 71, 19, 0, 8, 85, 19, 35, 113, 1, 7, 43, 9, 20, 64, 0, 4, 172, 24, 39, 151, 1, 7, 125, 10, 19, 126, 0, 4
b_y        [1859, 1859, 179, 783, 783, 0, 84, 102, 240, 257, 616, 1, 183, 3536, 142, 256, 43, 0, 6, 614, 1169, 67, 123, 1, 951, 31, 19, 121, 64, 0, 4, 26, 27, 1018, 151, 1
******************************
epoch   29 batch number 341 loss is:  5.957709312438965
prediction [10, 303, 4, 149, 192, 0, 39, 45, 43, 26, 38, 1, 97, 19, 43, 47, 528, 0, 9, 435, 480, 47, 123, 1, 7, 129, 9, 8, 193, 0, 10, 48, 10, 125, 46, 1, 7, 311, 36, 18,
b_y        [1450, 384, 4, 51, 286, 0, 941, 82, 4, 51, 151, 1, 27, 489, 905, 110, 528, 0, 731, 2338, 221, 54, 35, 1, 1156, 281, 10, 216, 2526, 0, 6, 21, 337, 284, 46, 1, 29
******************************
epoch   29 batch number 342 loss is:  6.095395565032959
prediction [10, 167, 66, 74, 85, 0, 17, 186, 264, 25, 6, 1, 97, 41, 5, 162, 29, 0, 9, 109, 4, 175, 5, 1, 7, 21, 7, 17, 177, 0, 7, 53, 34, 15, 38, 1, 72, 74, 35, 12, 1091,
b_y        [318, 318, 201, 74, 85, 0, 375, 186, 261, 60, 13, 1, 110, 1295, 115, 283, 28, 0, 622, 713, 1187, 770, 471, 1, 8, 196, 200, 54, 707, 0, 7, 344, 99, 123, 714, 1,
******************************
epoch   29 batch number 343 loss is:  6.126489639282227
prediction [10, 83, 179, 71, 48, 0, 22, 115, 4, 437, 211, 1, 293, 123, 141, 149, 8, 0, 7, 74, 311, 9, 7, 1, 7, 193, 230, 119, 177, 0, 119, 119, 77, 85, 5, 1, 7, 28, 12, 42
b_y        [1279, 1279, 1310, 154, 72, 0, 206, 206, 43, 293, 1497, 1, 2085, 3029, 452, 209, 8, 0, 60, 237, 92, 546, 13, 1, 1219, 983, 7, 14, 177, 0, 220, 75, 964, 145, 76,
******************************
epoch   29 batch number 344 loss is:  6.067944526672363
prediction [10, 7, 47, 44, 45, 0, 6, 451, 10, 132, 131, 1, 10, 57, 50, 4, 29, 0, 7, 119, 17, 17, 56, 1, 7, 32, 4, 92, 117, 0, 12, 16, 77, 9, 141, 1, 72, 48, 9, 9, 188, 0,
b_y        [196, 330, 69, 44, 70, 0, 581, 451, 18, 714, 457, 1, 8, 169, 5, 24, 73, 0, 6, 40, 100, 54, 626, 1, 185, 11, 32, 1148, 1148, 0, 249, 16, 63, 236, 236, 1, 118, 48
******************************
epoch   29 batch number 345 loss is:  6.184462070465088
prediction [10, 48, 22, 45, 70, 0, 39, 8, 19, 5, 7, 1, 4, 6, 10, 7, 124, 0, 10, 37, 36, 147, 125, 1, 7, 18, 16, 6, 16, 0, 7, 20, 4, 47, 12, 1, 6, 16, 4, 20, 124, 0, 4, 79,
b_y        [457, 152, 475, 147, 70, 0, 223, 8, 37, 313, 1117, 1, 302, 457, 231, 652, 1824, 0, 222, 704, 70, 147, 5, 1, 457, 48, 84, 88, 16, 0, 201, 72, 1766, 372, 1209, 1,
******************************
```

①何时不得去，不是不相随。

②不见天台行，何人是日光。风光生白露，风动柳条风。

③风光万里月，风动九重城。

④风光入夜夜，清露落花前。

⑤湖上白日日，一夜一重重。

⑥风雨无人识，风光入石门。

⑦湖上白日日，一夜一重重。风雨无人识，风光入石门。

⑧君子在人间，不得千门雪，一时终不见。风光生白发，月色动秋光。

五、总结与反思

本实验整体实现了一个字符级的自回归生成流程：从原始诗歌文本出发，完成字符表构建与序列化表示，通过嵌入把离散符号映射到连续空间，再用门控循环网络聚合上下文并在每个时间步预测下一字符的概率分布，最终在推理阶段用逐步续写直到终止的方式生成诗句。从结果层面看，这类方法通常能较快学到局部搭配与常见句式，生成文本在字面上会出现一定的连贯性与像诗的格式特征，但更深层的主题一致性、意象延续与结构约束（例如严格格律）往往难以稳定满足，反映出纯粹基于统计相关性的语言模型在可控性与全局规划能力上的天然不足。

字符级不需要分词，覆盖率高且实现简单，适合中文古诗这种短文本；但它把语义单位切得过细，模型需要用更多步数才能表达一个词或意象，长期依赖压力更大，也更容易生成看似通顺但语义漂移的内容。语料清洗与过滤规则对生成质量影响很直接：过滤过严会导致多样性不足，过滤过松会把噪声模式学进去；开始/结束标记的设计决定了模型对“何时收束”是否敏感。字符表的构建若不处理低频字、空白符号与标点的策略，常会导致生成中出现异常字符、重复标点或无意义填充，这些现象往往不是模型不会写，而是数据分布与符号体系本身在牵引输出。

门控循环网络相较普通循环网络更容易学习较长上下文，但训练仍可能出现梯度不稳定与收敛速度慢的问题，因此学习率、优化器选择、梯度裁剪阈值等超参数会显著影响最终效果。另一个常见现象是训练损失下降并不必然带来更好读的生成文本，因为损失主要衡量一步预测的平均正确性，而诗歌质量涉及更高层的结构与审美指标。若训练方式等价于让模型在每个时间步都尽量像训练集的下一个字符，模型会倾向于输出高频、稳妥的字，导致内容趋同、缺少新意，这也是最大似然训练在生成任务中的常见副作用。

如果始终选择概率最大的下一个字符，生成会更稳定但更保守，容易出现模板化和重复；若引入随机采样、温度系数或 top-k/top-p 采样，多样性会提升，但也更容易产生语义跳跃或不通顺。重复与循环是自回归模型的典型问题，尤其在字符级上更明显；实际可通过重复惩罚、禁止短周期重复、或在概率分布上做约束来缓解。终止条件也很关键：仅依赖模型输

出结束标记有时会失败，需要配合最大长度与格式约束，才能保证输出可用。

综合总结来看，本实验验证了“序列模型 + 条件概率逐步生成”这一范式能够在较小成本下学到文本的局部统计结构，并具备可运行的端到端生成能力；但要让生成更符合诗歌的整体结构与语义连贯，需要在数据层加入更明确的结构信号（如句长、标点模式、题材标签），在模型层引入更强的上下文建模（如注意力或更现代的序列建模架构），在解码层使用更合理的采样与约束策略，并建立更贴近目标的评价方法（人工评测结合困惑度、重复率、格式匹配率等）。这些改进方向的共同点是：不仅要拟合下一个字符，还要在训练与生成过程中显式地把可控性、全局一致性、结构约束纳入系统目标。